

UNIVERSIDADE FEDERAL DE ALAGOAS – UFAL

Campus A. C. Simões

Ciência da Computação

JOSÉ MILTON DE MORAES SILVA NETO

RELATÓRIO DO MILESTONE 2

Pontos Fortes e Pontos Fracos

Pontos Fortes

1. Arquitetura em Camadas

○ **Separação de Responsabilidades:**

Serviços como `UsuarioService`, `ComunidadeService` e `RelacionamentoService` isolam as regras de negócio, seguindo o padrão **Service Layer**. O `DataRepository` centraliza o acesso aos dados, aderindo ao padrão **Repository**, facilitando a manutenção e os testes.

○ **Facade Simplificada:**

A classe `Facade` atua como uma interface unificada, reduzindo a complexidade para o cliente e encapsulando interações com os serviços internos.

2. Reutilização de Componentes

- Os serviços compartilham o `DataRepository`, evitando duplicação de código e centralizando a lógica de acesso aos dados.

3. Extensibilidade

- A modularização permite adicionar novos recursos (como tipos de relacionamentos) sem impactar os serviços existentes.

Pontos Fracos

1. Acoplamento entre Serviços

- **Dependências Circulares:** Serviços como `MensagemService` dependem de `UsuarioService` e `ComunidadeService`, o que pode dificultar testes unitários e aumentar a fragilidade do sistema.

2. Gestão de Sessões

- **Complexidade Ocultas:** A lógica de sessão está dividida entre `SessaoService` e `DataRepository`, dificultando o rastreamento de fluxos.

3. Desempenho em Escala

- **Estruturas de Dados Não Otimizadas:** O uso de `LinkedList` para mensagens e `HashMap` para usuários pode ser ineficiente em cenários com milhares de registros.

Padrões de Projeto Utilizados

1. Padrão Service Locator

1.1 Definição e Propósito

- O **Service Locator** é um padrão que encapsula os processos de obtenção de serviços, fornecendo uma abstração centralizada para gerenciar dependências.

1.2 Implementação no Sistema Jackut

- No sistema, o padrão foi implementado pela classe `ServiceLocator`.

1.3 Funcionamento no Sistema

- **Inicialização:** O `ServiceLocator` é inicializado ou carregado de um arquivo na classe `Facade`.
- **Acesso aos Serviços:** A `Facade` obtém referências aos serviços por meio do `ServiceLocator`.
- **Persistência:** O `ServiceLocator` é serializado para manter o estado do sistema.

1.4 Benefícios e Desafios

- **Vantagens:** Oculta dependências e centraliza o gerenciamento de serviços.
- **Desvantagens:** Pode complicar testes unitários e violar o Princípio de Responsabilidade Única.

2. Padrão Service Layer

2.1 Definição e Propósito

- O **Service Layer** define uma camada que encapsula a lógica de negócio, separando-a da interface do usuário e do acesso a dados.

2.2 Implementação no Sistema Jackut

- Foi implementado por classes como `UsuarioService`, `SessaoService`, `ComunidadeService`, `MensagemService` e `RelacionamentoService`, cada uma responsável por um domínio específico.

2.3 Funcionamento no Sistema

- **Separação de Responsabilidades:** Cada serviço gerencia sua própria lógica.
- **Coordenação:** Os serviços colaboram para operações complexas.
- **Acesso a Dados:** Utilizam o `DataRepository` para manipular dados.

2.4 Benefícios

- **Reutilização de Código:** Lógica encapsulada em serviços reutilizáveis.
- **Testabilidade e Manutenibilidade:** Facilita testes e manutenção.

3. Interação entre Service Locator e Service Layer

- O `ServiceLocator` gerencia as instâncias dos serviços.
- O **Service Layer** (classes de serviço) encapsula a lógica de negócio.
- A `Facade` usa o `ServiceLocator` para acessar os serviços.

Organização do Projeto

O projeto **JACKUT** está estruturado em três camadas principais:

1. Camada de Entidades:

- Modelos como `Comunicacao`, `Comunidade`, `Relacionamento` e `Usuario`, focados em armazenar dados.

2. Camada de Serviços:

- Classes como `DataRepository`, `SessaoService`, `MensagemService`, etc., responsáveis pela lógica de negócio.

3. Camada de Integração:

- **Facade:** Interface simplificada para interações externas.
- **ServiceLocator:** Centraliza a criação e acesso aos serviços.

Essa estrutura garante modularidade e facilidade de manutenção.